

Fundamentals of Operating System Design (CEUC301)

Buddy Memory Allocation

Prashant Jadav
DEPSTAR, CHARUSAT

Learning Outcomes

- Understand dynamic kernel memory allocation.
- Explain external and internal fragmentation.
- Describe the motivation behind the buddy allocator.
- Understand power-of-two allocation.
- Represent buddy memory as a binary tree.

Why Do We Need a Better Allocator?

Applications continuously request and release memory.

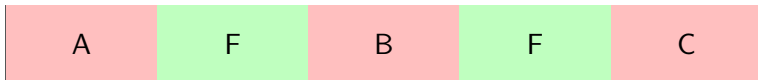


Problems:

- Memory becomes fragmented.
- Large requests may fail despite sufficient total free memory.
- Allocation must be fast.

External Fragmentation

Example:



Although 4 MB is free ($2\text{ MB} + 2\text{ MB}$), a request for 4 MB cannot be satisfied because the free space is not contiguous.

Internal Fragmentation

Suppose requests are rounded to fixed sizes.

Request	Allocated	Waste
10 KB	16 KB	6 KB
18 KB	32 KB	14 KB
70 KB	128 KB	58 KB

The buddy allocator trades a small amount of internal fragmentation for very fast allocation.

Idea Behind the Buddy Allocator

Rules:

- ① Total memory size is a power of two.
- ② Every allocated block is a power of two.
- ③ If required, a larger block is recursively split.
- ④ On deallocation, buddies are merged.

Example Memory

Assume kernel has 1024 KB.



1024 KB Free Block

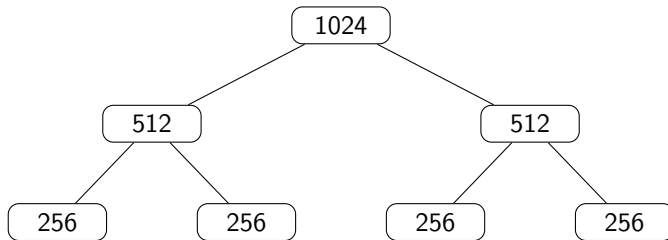
If a process requests 200 KB, allocating the whole 1024 KB would waste memory.

Power-of-Two Allocation

Requested	Allocated
20 KB	32 KB
70 KB	128 KB
130 KB	256 KB
300 KB	512 KB
700 KB	1024 KB

Always allocate the smallest power-of-two block that can satisfy the request.

Binary Tree Representation



Each node can be split into two equal buddies.

Allocation Algorithm

Request size R :

- 1 Round R to next power of two.
- 2 Find smallest free block.
- 3 If unavailable, split a larger block.
- 4 Continue until desired size is obtained.
- 5 Mark block as allocated.

Initial Memory

1024 KB Free

Request 100 KB

Rounded to 128 KB.

$1024 \rightarrow 512 + 512$

$512 \rightarrow 256 + 256$

$256 \rightarrow 128 + 128$

128 A

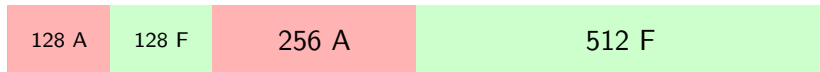
128

256

512

Request 200 KB

Rounded to 256 KB.



Request 300 KB

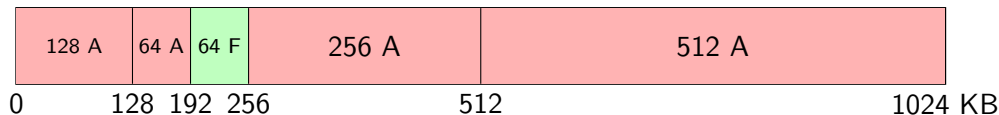
Rounded to 512 KB.



Request 50 KB

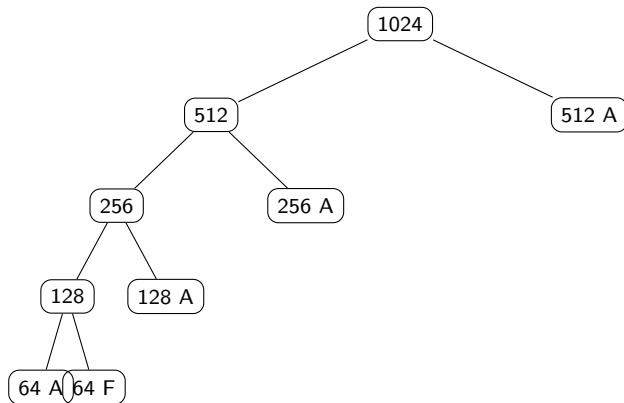
Need 64 KB.

The remaining **free 128 KB block** is split into two **64 KB buddy blocks**.



- The first 128 KB block remains allocated.
- The second 128 KB block is split into two 64 KB buddies.
- The left buddy is allocated to satisfy the 50 KB request.
- The right buddy remains free.

Allocation Tree



Observations

- Blocks are always powers of two.
- Only the required branch is split.
- Other blocks remain untouched.
- Allocation complexity is approximately $O(\log n)$.
- Internal fragmentation may occur due to rounding.

Key Observations

- Buddy allocator uses divide-and-conquer.
- Splitting continues until the required block size is obtained.
- Every block has exactly one buddy.
- Free buddies are merged during deallocation.
- Part 2 will cover allocation with detailed worked examples.

Memory State Before Deallocation

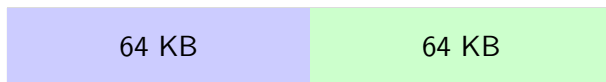


Buddy Property

Two blocks are buddies if:

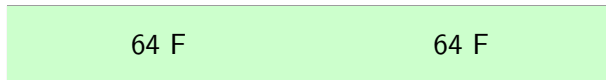
- They have the same size.
- They originate from the same split.
- They are adjacent.

Example:



These two 64 KB blocks are buddies.

Step 1: Free the 64 KB Block



Both buddies are free, so they can merge.

Step 2: Merge into 128 KB

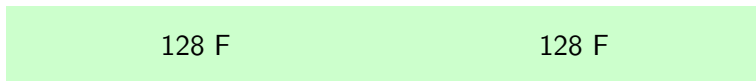


128 KB Free

The allocator replaces the two 64 KB buddies with one 128 KB block.

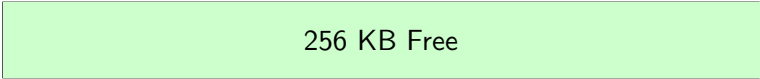
Step 3: Free Adjacent 128 KB

Now free the allocated 128 KB block.



Again, both buddies are free.

Step 4: Merge into 256 KB

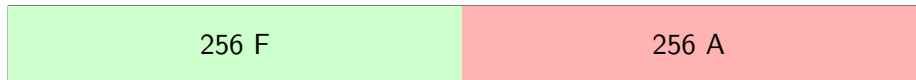


256 KB Free

The merge continues recursively whenever buddies are free.

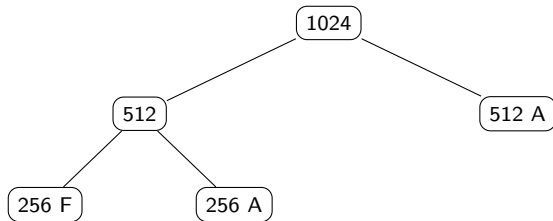
When Merging Stops

Suppose the buddy of the 256 KB block is still allocated.



Since both buddies are not free, merging stops.

Recursive Coalescing Example



Only the left subtree is free. No further merge is possible.

Buddy Address Calculation

For a block of size 2^k :

$$\text{Buddy Address} = \text{Block Address} \oplus \text{Block Size}$$

Example:

- Address = 256 KB
- Block Size = 256 KB
- Buddy = $256 \oplus 256 = 0$

The XOR operation quickly locates the buddy block.

Complete Example

Operation	Result
Allocate 100 KB	128 KB allocated
Allocate 200 KB	256 KB allocated
Allocate 300 KB	512 KB allocated
Allocate 50 KB	64 KB allocated
Free 64 KB	Merge to 128 KB
Free 128 KB	Merge to 256 KB
Merge stops	Buddy allocated

Key Points

- Deallocation itself is simple.
- Coalescing reduces external fragmentation.
- Merging is recursive.
- Buddy lookup is efficient using XOR.
- Worst-case merge complexity: $O(\log n)$.

Summary

- Motivation for buddy allocation
- External vs. internal fragmentation
- Power-of-two allocation
- Binary tree organization
- Foundation for splitting and merging

Slab Allocator

Kernel Memory Allocation

Prashant Jadav

DEPSTAR - CHARUSAR

Learning Objectives

By the end of this lecture, students should be able to:

- Explain the motivation for the slab allocator.
- Distinguish cache, slab, object, and page.
- Explain object allocation and deallocation.
- Describe full, partial, and empty slabs.
- Explain object reuse and constructor/destructor concepts.
- Analyze fragmentation and allocation overhead.
- Compare slab allocation with the buddy system.
- Solve numerical and HOT-style questions.

Why Do Operating Systems Need a Slab Allocator?

Kernel code frequently allocates and frees objects of known types:

- Process/thread control structures
- File-system objects
- Inodes
- Network buffers
- Device-related structures

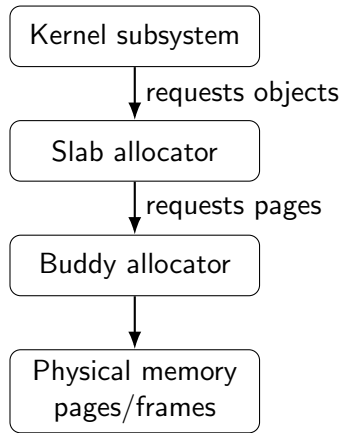
Repeatedly obtaining memory directly from a general-purpose allocator can cause:

- allocation/deallocation overhead,
- fragmentation,
- repeated object initialization,
- poor cache locality.

Core idea

Keep groups of pre-allocated objects ready for rapid reuse.

Where Does the Slab Allocator Fit?



Slab allocator typically sits above a page-level allocator

Basic Terminology

Cache: Collection of slabs dedicated to one object type/size.

Slab: One or more contiguous pages containing multiple objects.

Object: Individual allocation unit inside a slab.

Free object: Object currently available for allocation.

Partial slab: Slab containing both allocated and free objects.

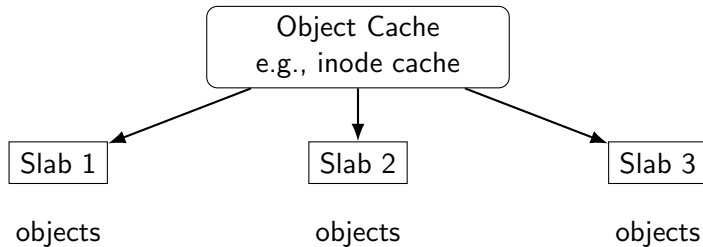
Full slab: All objects are allocated.

Empty slab: All objects are free.

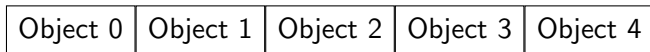
Example

A cache for `task_struct` contains slabs, and each slab contains multiple `task_struct` objects.

Slab Allocator Hierarchy



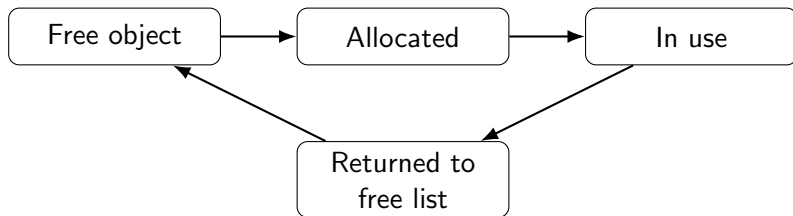
Slab: pages allocated from lower-level memory allocator



Fixed-size objects of one cache

One slab contains many objects of the same cache type

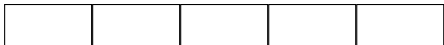
Object Lifecycle



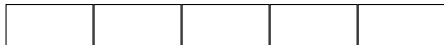
The important optimization is:

reuse an existing object instead of repeatedly constructing memory from scratch

Free, Partial and Full Slabs



Partial slab
allocated + free



Full slab
all allocated

A cache commonly keeps track of slabs according to their occupancy state.

Allocation Operation

When a kernel subsystem requests an object:

- ① Identify the appropriate cache.
- ② Find a slab containing a free object.
- ③ Remove one object from the free list.
- ④ Mark/use the object.
- ⑤ Return its address.

If no free object exists

A new slab is obtained from the lower-level page allocator and divided into objects.

Cache lookup → free object → return address

Deallocation Operation

When an object is released:

- ① Return the object to its cache.
- ② Put it back on the slab's free list.
- ③ Update slab occupancy information.
- ④ Optionally reclaim completely empty slabs.

Important

The physical pages need not be immediately returned to the system. They may be retained so that future allocations can be satisfied quickly.

Why Object Reuse Improves Performance

Without a slab cache:

allocate pages → construct object → use

Repeatedly:

free → allocate → initialize

With a slab cache:

reuse prepared object → use

Benefits:

- reduced allocation overhead,
- reduced initialization cost,
- improved cache locality,
- predictable object sizes,
- fast deallocation.

Constructors and Destructors

For complex kernel objects, initialization may be expensive.

A slab cache can support a constructor that prepares a newly created object.

New slab → construct objects → objects reused repeatedly

On object reuse, the allocator can avoid repeating some initialization work.

Important distinction

A constructor associated with a slab cache is not the same as a C++ constructor. It is an allocator/kernel mechanism for preparing objects.

Numerical Example 1: Objects per Slab

Suppose:

Page size = 4096 bytes

Slab = 2 pages

Object size = 256 bytes

Slab capacity:

$$2 \times 4096 = 8192 \text{ bytes}$$

Number of objects:

$$\frac{8192}{256} = \boxed{32}$$

Therefore, one slab can hold:

32 objects

ignoring metadata/alignment overhead.

Numerical Example 2: Slabs Required

A cache needs to accommodate:

100 objects

Each slab contains:

24 objects

Number of slabs:

$$\left\lceil \frac{100}{24} \right\rceil = \boxed{5}$$

Objects physically available:

$$5 \times 24 = 120$$

Unused object slots:

$$120 - 100 = \boxed{20}$$

This unused capacity is different from external fragmentation: the allocator has deliberately organized fixed-size object slots within slabs.

Numerical Example 3: Memory Utilization

Suppose:

Object size = 200 B

Slab size = 4096 B

Objects per slab:

$$\left\lfloor \frac{4096}{200} \right\rfloor = 20$$

Object storage:

$$20 \times 200 = 4000 \text{ B}$$

Unused space:

$$4096 - 4000 = \boxed{96 \text{ B}}$$

Object-space utilization:

$$\frac{4000}{4096} \times 100 \approx \boxed{97.66\%}$$

Metadata and alignment can reduce the actual utilization further.

Slab Metadata Overhead

Suppose:

$$\text{Slab size} = 8192 \text{ B}$$

$$\text{Object size} = 256 \text{ B}$$

$$\text{Metadata} = 128 \text{ B}$$

If metadata occupies part of the slab:

$$\text{usable space} = 8192 - 128 = 8064$$

Objects:

$$\left\lfloor \frac{8064}{256} \right\rfloor = \boxed{31}$$

Object space:

$$31 \times 256 = 7936 \text{ B}$$

Remaining:

$$8064 - 7936 = \boxed{128 \text{ B}}$$

Slab Allocator and Fragmentation

External fragmentation:

- Slab allocation uses pages obtained from a lower-level allocator.
- Fixed-size objects avoid variable-size holes between objects.

Internal/wasted space:

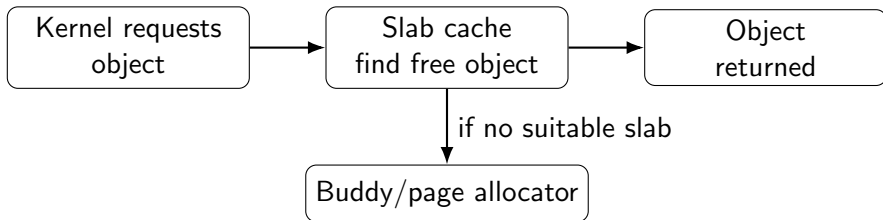
- Last portion of a slab may not fit another complete object.
- Alignment may create padding.
- Metadata consumes space.

Slab allocation reduces object-level fragmentation, but does not mean zero wasted memory

Buddy System vs Slab Allocator

Property	Buddy System	Slab Allocator
Allocation unit	Blocks of power-of-two pages	Objects within slabs
Best suited for	Page/block allocation	Frequently used kernel objects
Object size	Variable power-of-two blocks	Cache-specific fixed-size objects
Fragmentation	Internal possible	Internal/alignment overhead possible
Speed	Fast block allocation	Very fast object allocation
Reuse	General blocks	Specialized object reuse
Relationship	Lower-level allocator	Often built above page allocator

Allocation Path: Buddy + Slab



Buddy system manages larger page blocks; slab manages reusable objects

Advantages of Slab Allocation

- Very fast allocation/deallocation for common object sizes.
- Excellent object reuse.
- Reduces repeated initialization.
- Good cache locality.
- Reduces allocation overhead for frequently used kernel objects.
- Object-specific caches can be optimized independently.
- Avoids the need for general-purpose searches for every object request.

Disadvantages of Slab Allocation

- Memory can remain reserved in caches even when demand temporarily falls.
- Cache metadata consumes memory.
- Alignment and object-size constraints may waste space.
- More complex than a simple free-list allocator.
- Too many specialized caches can increase management overhead.
- Cache behavior and synchronization become important on multicore systems.

Multicore Considerations

On a multicore system, many CPUs may allocate/free objects simultaneously.
A single global lock can cause:

contention

Modern allocators therefore commonly use mechanisms such as:

- per-CPU caches,
- local free lists,
- batching,
- reduced lock contention.

Performance idea

Keep frequently allocated objects close to the CPU that uses them when practical.

Real-World Analogy: Hotel Housekeeping

Think of a hotel:

- **Cache** = collection of rooms of one category.
- **Slab** = a group/floor containing several similar rooms.
- **Object** = one room.
- **Allocation** = assigning a free room to a guest.
- **Deallocation** = guest leaves and room becomes reusable.

Instead of constructing a room for every guest:

prepare once, reuse many times

This captures the central performance idea of slab allocation.

Common Exam Questions

- ① What problem does the slab allocator solve?
- ② Distinguish cache, slab and object.
- ③ What happens when all objects in a slab are allocated?
- ④ Why does slab allocation improve performance?
- ⑤ What is the role of the buddy allocator?
- ⑥ Calculate objects per slab.
- ⑦ Calculate the number of slabs required.
- ⑧ Calculate unused space due to object-size mismatch.
- ⑨ Compare slab allocation with buddy allocation.
- ⑩ Explain why slab allocation does not imply zero memory waste.

HOT Question 1

A kernel repeatedly allocates objects of size 192 bytes. Pages are 4096 bytes and a slab consists of 2 pages.

Tasks:

- 1 Calculate the maximum number of objects per slab.
- 2 Calculate unused space.
- 3 Explain how alignment could change the result.
- 4 Explain why a slab cache can outperform repeated general-purpose allocation.

Expected calculation:

$$\left\lfloor \frac{8192}{192} \right\rfloor = \boxed{42}$$

$$8192 - (42 \times 192) = \boxed{128 \text{ B}}$$

before metadata/alignment overhead.

HOT Question 2

Two kernel subsystems require:

Subsystem	Object size
<i>A</i>	128 B
<i>B</i>	1024 B

Both use 4-KB slabs.

Question: Should both object types share one cache?

Discuss:

- internal space utilization,
- object reuse,
- cache locality,
- metadata,
- allocation speed.

Caches are generally organized around compatible object types/sizes rather than mixing unrelated allocation patterns.

HOT Question 3: Design Decision

A kernel has:

- 1 million allocations per second,
- objects of fixed size 256 B,
- very short object lifetimes.

Compare:

General allocator vs slab cache

Discuss:

- ① allocation latency,
- ② initialization cost,
- ③ locality,
- ④ memory retention,
- ⑤ synchronization on multicore systems.

Expected reasoning

It should justify the design using workload characteristics.

Numerical Practice Problem

A slab cache uses:

page size = 4096 B

slab order = 2

object size = 320 B

metadata = 128 B

- 1 Find slab size,
- 2 usable bytes,
- 3 objects per slab,
- 4 unused bytes after placing the objects.

Solution:

$$\text{Slab size} = 2^2 \times 4096 = \boxed{16384 \text{ B}}$$

$$\text{Usable} = 16384 - 128 = 16256$$

$$\text{Objects} = \left\lfloor \frac{16256}{320} \right\rfloor = \boxed{50}$$

Cache → Slabs → Objects

- Cache = collection for a particular object type/size.
- Slab = one or more pages containing objects.
- Object = actual allocation unit.
- Allocation = take a free object.
- Deallocation = return object to its cache.
- Empty slabs may be reclaimed.
- Buddy allocator can supply pages to the slab allocator.

Key Takeaways

- ① Slab allocation is designed for **frequent allocation of kernel objects**.
- ② It improves performance through **object reuse**.
- ③ A cache contains slabs, and slabs contain objects.
- ④ Empty/partial/full slab states help manage available objects.
- ⑤ Slab allocation complements, rather than necessarily replaces, a lower-level page allocator.
- ⑥ The major trade-off is:

Fast object allocation + reuse $\iff$ cache/metadata memory overhead